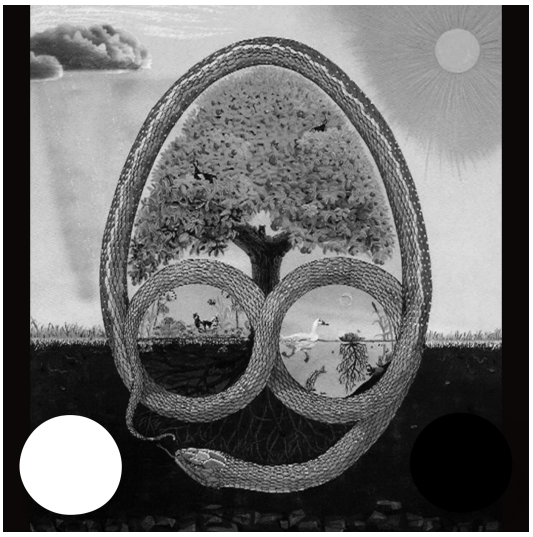


Merkle Providence Reverse RAG

Provenance-Preserving Cache Chains for Small Language Models

How Merkle trees turn Reverse RAG into a verifiable, shared knowledge layer.

How 7B models answer with the confidence of a verified record, not a guess.

russell@unturf, cthegray, TimeHexOn, foxhop

unfirehose.com · uncloseai.com · permacomputer.com

April 2026

Abstract

License: AGPL-3.0-only · This algorithm, its implementation, & all associated code carry the GNU Affero General Public License v3.0 (only). You may use, modify, & distribute under those terms. No proprietary relicensing exists.

Reverse Retrieval Augmented Generation (Reverse RAG) demonstrated that client-side context injection lets small 7B-8B parameter language models outperform much larger models on page-specific questions. Our prior work showed that the client already holds the document. No vector database required. No embedding pipeline. No retrieval failures.

Merkle Providence Reverse RAG extends this with a second insight: the client not only holds the document, it can *remember what it already computed about that document*, prove that memory came from an unmodified source, & share that verified knowledge with others on public or private chains.

A Merkle tree over document content chunks produces a root hash: a 32-byte fingerprint that changes when any part of the document changes. Pairing this root hash with a question hash yields a cache key. A cache hit means a previously computed answer returns instantly, with a Merkle proof of origin. A cache miss triggers fresh inference, stores the result, & extends our chain.

Unfirehose serves as our chain substrate. Every inference session produces JSONL records that feed into SQLite at `~/.unfirehose/unfirehose.db`. Public chains broadcast verified answer records to peers. Private chains stay local. Either way, a small model gains access to a growing body of pre-verified answers, each carrying cryptographic proof of the document version that produced it.

This combination removes two limits that have constrained small models:

1. **The repetition tax**: answering the same question about the same document on every visit, burning inference tokens & time, producing answers that may vary slightly each run.
2. **The trust deficit**: small models hallucinate. Cached answers with Merkle proofs do not hallucinate. They either match a verified record or they do not exist yet.

Once published, this technique forces a reckoning: every RAG system that cannot prove provenance of its retrieved chunks operates on unverifiable context. Merkle Providence makes provenance a first-class primitive.

1. The Problem Reverse RAG Left Open

Reverse RAG solved context retrieval. Client-side DOM extraction replaced vector databases. Full-page injection replaced chunk fragmentation. Small models with perfect context outperformed large models with partial context.

One problem remained: **repetition**.

A user visits a technical document. A 7B model reads 8,000 tokens of page content, classifies the page, & answers their question. One hour later, a different user visits the same document. Our same 7B model reads our same 8,000 tokens, classifies our same page, & answers a similar question again. Tomorrow, a third user. Our same 8,000 tokens. Our same classification. Our same inference cycle.

Nothing remembered. Nothing reused. No proof that any two answers came from our same source.

This repetition carries three costs:

- **Compute cost**: every visit re-runs inference on content that has not changed.
- **Consistency cost**: stochastic inference on identical inputs produces slightly different answers. Two users asking near-identical questions may receive meaningfully different responses from our same page.
- **Trust cost**: no mechanism exists to verify that an answer actually derived from a specific document version. A model can claim anything.

Merkle Providence addresses all three.

2. Merkle Trees as Content-Addressed Cache Keys

A Merkle tree splits a document into chunks, hashes each chunk, then builds a binary tree of hashes upward until a single root hash remains. This root hash has two properties that make it ideal for caching:

Property 1: Determinism. Our same document always produces our same root hash. Two clients processing our same page independently compute identical roots. No coordination required.

Property 2: Tamper sensitivity. Any change to any byte of the document changes at least one leaf hash, which propagates upward to invalidate our root. A document that changed since our last visit produces a new root hash automatically. Our cache never serves a stale answer for a modified document.

Cache key construction:

```
document_root = merkle_root(chunks(page_content))
question_key  = sha256(normalize(user_question))
cache_key     = document_root + ":" + question_key
```

A cache hit at this key means: *an earlier session computed an answer to this question, about this exact version of this document.* Our answer arrives in O(1) time. Our Merkle proof confirms our document version.

A cache miss means: run inference, store our result, extend our chain.

3. The Providence Layer: Proof of Origin

"Providence" carries two meanings that both apply here:

Provenance: where something came from. A Merkle proof lets any verifier confirm that an answer derived from a specific document version without receiving our full document. Our proof path, a sequence of sibling hashes from leaf to root, constitutes a compact cryptographic certificate of origin.

Foresight: the system appears to "already know" answers to questions asked before. From a user's perspective, a cache hit looks like instant comprehension. Our model responded before seemingly finishing to read. This resembles foresight because our prior sessions did our work ahead of time.

A providence record for a cached answer contains:

```
{
  "cache_key":      "sha256:abc123...:sha256:def456...",
  "document_root":  "sha256:abc123...",
  "document_uri":   "https://example.com/docs/api",
  "question_hash":  "sha256:def456...",
  "question_text":  "What does the rate_limit field mean?",
  "answer_text":    "The rate_limit field specifies...",
  "merkle_proof":   ["sha256:111...", "sha256:222...", ...],
  "model":          "hermes-3-llama-3.1-8b",
  "timestamp":      "2026-04-06T18:00:00Z",
  "chain":          "public"
}
```

Any recipient of this record can verify our Merkle proof against our document root. If our document root matches what they compute from our same URI, our answer carries full provenance. If their computation produces a different root, our document changed, our answer no longer applies to their version.

4. Unfirehose as Chain Substrate

Unfirehose collects JSONL session data from machine learning harnesses across a mesh of compute nodes. Every tool call, every model response, every session event flows into `~/.unfirehose/unfirehose.db` via our background ingestion worker.

Our Merkle providence layer extends this existing infrastructure with a providence cache table:

```
CREATE TABLE providence_cache (
  cache_key        TEXT PRIMARY KEY,
  document_root    TEXT NOT NULL,
  document_uri     TEXT NOT NULL,
  question_hash    TEXT NOT NULL,
  question_text    TEXT NOT NULL,
  answer_text      TEXT NOT NULL,
  merkle_proof     TEXT NOT NULL,  -- JSON array of sibling hashes
  model            TEXT NOT NULL,
  created_at       INTEGER NOT NULL,
  chain            TEXT NOT NULL DEFAULT 'private',
  peer_count       INTEGER NOT NULL DEFAULT 0
);

CREATE INDEX idx_providence_root ON providence_cache(document_root);
CREATE INDEX idx_providence_uri  ON providence_cache(document_uri);
```

Two chain modes:

Private chain: records stay local at `~/.unfirehose/`. Our personal session history. Our model learns from our own prior work on documents we visit repeatedly. Our chain belongs to us alone.

Public chain: records broadcast to peers via our unfirehose router daemon. Any node on our mesh that has processed our same document root can contribute answers. Any node that queries our same document root can receive pre-verified answers from others. Our chain grows with every participant.

Public chains do not share document content. They share proofs. Our Merkle proof proves an answer came from a specific document version without transmitting our document. Document content stays on our originating client, exactly as Reverse RAG intended.

5. How Small Models Punch Outside Their Weight Class

Parameter count governs a model's ability to reason from first principles. Context quality governs a model's ability to answer questions about specific documents. Verified memory governs a model's ability to give consistent, trustworthy answers over time.

Large models win on reasoning from first principles. Small models, given the right tools, win on everything else.

Merkle Providence Reverse RAG gives small models three tools:

Tool 1: Full context (from Reverse RAG). Our same page injection described in our prior paper. Our model receives every word, every heading, every entity, every code block of our document. Nothing retrieved by cosine similarity. Nothing fragmented. Our full document, live from our DOM.

Tool 2: Verified recall (from our providence cache). Before our model reads our document, we check our cache. A cache hit returns our answer in milliseconds with a Merkle proof. Our model delivers a verified answer faster than any large model can load our full context. Our small model "already knew" because our prior session did our work.

Tool 3: Peer-verified answers (from public chains). On a public chain, our small model benefits from every session any participant ran against our same document version. A document processed by 100 sessions across our mesh produces a rich cache. Our 7B model answers from our collective record, not just its own prior work.

These three tools operate at different timescales:

- **O(1)**: cache hit, local or peer. Instant. No inference required.
- **O(context)**: Reverse RAG with full page injection. Fast. One inference pass over complete context.
- **O(reasoning)**: fresh inference on novel questions. Standard. One inference pass, uncached.

A large model operates exclusively at O(reasoning). Our small model with Merkle Providence operates at O(1) for previously answered questions, O(context) for new questions on known documents, & O(reasoning) only for truly novel questions on new documents.

6. Architecture

Stage 1: Document Fingerprinting

Our same DOM extraction from Reverse RAG runs first. Once our extraction completes, we split our content into 512-token chunks & build our Merkle tree:

```
function merkleRoot(chunks) {
    let layer = chunks.map(c => sha256(c));
    while (layer.length > 1) {
        const next = [];
        for (let i = 0; i < layer.length; i += 2) {
            const left  = layer[i];
            const right = layer[i + 1] ?? left;  // odd node doubles itself
            next.push(sha256(left + right));
        }
        layer = next;
    }
    return layer[0];
}
```

Our root hash serves as our document fingerprint for our entire session. One hash computation. No server calls.

Stage 2: Cache Lookup

Before any LLM call, we check our providence cache:

```
async function lookupProvidence(documentRoot, questionText) {
    const questionHash = sha256(normalize(questionText));
    const cacheKey     = `${documentRoot}:${questionHash}`;

    // Check local chain first
    const local = await db.get(
        'SELECT * FROM providence_cache WHERE cache_key = ?',
        [cacheKey]
    );
    if (local) return { answer: local.answer_text, proof: local.merkle_proof, source: 'local' };

    // Check public chain peers
    if (publicChainEnabled) {
        const peer = await queryPeers(cacheKey);
        if (peer && verifyMerkleProof(peer.merkle_proof, documentRoot)) {
            await storeLocal(peer);  // cache locally for future hits
            return { answer: peer.answer_text, proof: peer.merkle_proof, source: 'peer' };
        }
    }

    return null;  // cache miss, proceed to inference
}
```

Stage 3: Inference & Cache Write

On a cache miss, our standard Reverse RAG inference runs. Once our answer arrives, we store our providence record:

```
async function storeProvidence(documentRoot, documentUri, question, answer, model) {
    const questionHash = sha256(normalize(question));
    const cacheKey     = `${documentRoot}:${questionHash}`;
    const merkleProof  = computeProof(documentRoot, question);  // path to root

    await db.run(`
        INSERT OR REPLACE INTO providence_cache
        (cache_key, document_root, document_uri, question_hash, question_text,
         answer_text, merkle_proof, model, created_at, chain)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    `, [cacheKey, documentRoot, documentUri, questionHash, question,
        answer, JSON.stringify(merkleProof), model, Date.now(), chainMode]);

    if (chainMode === 'public') broadcastToMesh(cacheKey);
}
```

Stage 4: Proof Verification

Any party that receives a providence record can verify it:

```
function verifyMerkleProof(proof, claimedRoot, leafContent) {
    let current = sha256(leafContent);
    for (const sibling of proof) {
        current = sha256(current < sibling
            ? current + sibling
            : sibling + current);
    }
    return current === claimedRoot;
}
```

If our document root computed from our verifier's live DOM matches `claimedRoot`, our answer carries full provenance. Our answer derives from our exact document version our verifier currently views. If our roots diverge, our document changed. Our cache entry no longer applies.

7. Public vs. Private Chains

Property	Private Chain	Public Chain
Storage	`~/.unfirehose/` local only	Broadcast via unfirehose router mesh
Participants	One user, one machine	Any node on our mesh
Document content shared	Never	Never (proofs only)
Answer content shared	Never	Yes, with Merkle proof
Trust model	Self-trust: our own prior work	Peer-trust: verified by Merkle proof
Best for	Personal document libraries, sensitive internal docs	Open documentation, public research, shared reference material
Cache growth rate	Linear in our own sessions	Linear in mesh participant sessions

Private chains suit corporate knowledge bases, personal research notes, any document our user does not want to share. Our local model benefits from our own prior sessions. No data leaves our machine.

Public chains suit open documentation, public APIs, research papers, any document where sharing verified answers benefits our community. No document content leaves our originating client. Proofs travel instead.

Our unfirehose router daemon manages chain membership & peer discovery. Our user opts in to public chains per-document-domain or per-session.

8. Forcing the Issue

Some technical formulations compel structural change not because they require adoption but because their existence makes our prior approach indefensible.

Our Merkle Providence layer forces one question onto every RAG system: *can you prove where your context came from?*

Traditional RAG retrieves chunks by cosine similarity. No chunk carries a Merkle proof. No retrieval carries a document fingerprint. Our retrieved context could come from a stale index, a corrupted chunk, or a document that changed since our last ingestion. Our RAG pipeline trusts our embedding similarity as a proxy for relevance. Nothing verifies our source.

Once Merkle-proved answers exist in our ecosystem, unverified RAG answers carry an implicit disclaimer: *I retrieved this from somewhere, I believe it matches your question, but I cannot prove our source was our document you currently view.*

Our providence cache answers carry a different posture: *I computed this answer from our exact bytes of our document whose root hash I can show you. Verify it yourself.*

This posture change does not require every system to adopt Merkle trees immediately. It requires every system to acknowledge that provenance matters, & that systems which cannot demonstrate provenance operate at a disadvantage in trust-sensitive contexts.

Medical documentation. Legal reference material. Security advisories. Financial disclosures. Any domain where "I retrieved something similar" falls short of "I can prove this came from our exact source you cited."

9. Unfirehose Integration

Unfirehose already collects JSONL from Claude Code, Fetch, & uncloseai harnesses across our mesh. Our providence cache extends this with three new API routes:

`GET /api/providence?uri=...`

Returns all cached answer records for a document URI. Includes document root hash, question hashes, answer previews, & peer counts. Lets users audit what our mesh knows about a document.

`POST /api/providence`

Stores a new providence record. Accepts `{ document_root, document_uri, question_text, answer_text, merkle_proof, model }`. Validates our proof before storing.

`GET /api/providence/peers?root=...`

Queries our mesh for cached records matching a document root hash. Returns answers from public-chain peers with their Merkle proofs. Our uncloseai.js client calls this before any inference to check for peer cache hits.

Our existing unfirehose dashboard gains a Providence page: a view of our cache landscape across documents, peer contribution rates, cache hit rates per domain, & our chain health.

10. Relationship to Zero-Knowledge Proofs

A Merkle proof constitutes a specific form of zero-knowledge argument: *I can prove this leaf exists in this tree without revealing our other leaves.*

In our context: *I can prove this answer came from this document version without revealing our full document.*

Our full ZKP apparatus (zk-SNARKs, Bulletproofs) provides stronger guarantees & more complex proofs. Merkle proofs provide a simpler, faster, & more practical subset: membership proofs sufficient for our provenance use case.

Our connection to ZKP deepens when our public chain handles sensitive documents. A user could prove their answer derived from a specific confidential document without revealing our document content, our question text, or our answer text. Only our proof path & our document root travel over our network. Our verifier confirms our proof without seeing our data.

This positions our public chain as opt-in verifiable knowledge sharing, not compelled disclosure.

11. Why This Matters for Permacomputer

Permacomputer infrastructure grows knowledge that outlasts our individual sessions, our individual machines, & our individual participants. Code seeds sprout on any abandoned technology. Knowledge should do our same.

A private chain that lives only on one machine dies with that machine. Our public chain on our unfirehose mesh survives node failures because our same document root on a different node still matches our cached proofs on our other nodes. Our knowledge persists as long as any mesh participant holds a copy.

Our Merkle structure aligns with permacomputer values:

Truth: proofs over assertions. Our answer carries a mathematical certificate, not just a model's confidence score.

Freedom: private chains for sensitive knowledge, public chains for shared knowledge. Our user chooses. No platform extracts our knowledge without consent.

Harmony: cache hits replace repeated inference. Our same compute produces more answers over time. Our mesh grows more efficient as our cache fills.

Love: small models with providence caches remove our infrastructure barrier. A 7B model on a consumer GPU, connected to our public chain, answers with our collective memory of our mesh. Our permacomputer does not require our most expensive hardware to deliver our most reliable answers.

12. Implementation Roadmap

Phase 1: Local private chain (unfirehose extension)

- Add `providence_cache` table to our unfirehose SQLite schema
- Extend our uncloseai.js Reverse RAG pipeline with pre-inference cache lookup & post-inference cache write
- Add `GET /api/providence` & `POST /api/providence` routes to our unfirehose web app
- Add Providence view to our dashboard

Phase 2: Public chain via unfirehose router

- Extend our router daemon with `/providence/broadcast` & `/providence/query` endpoints
- Add peer discovery to our mesh probe logic
- Add per-domain opt-in controls to our settings page
- Add peer contribution metrics to our Providence dashboard view

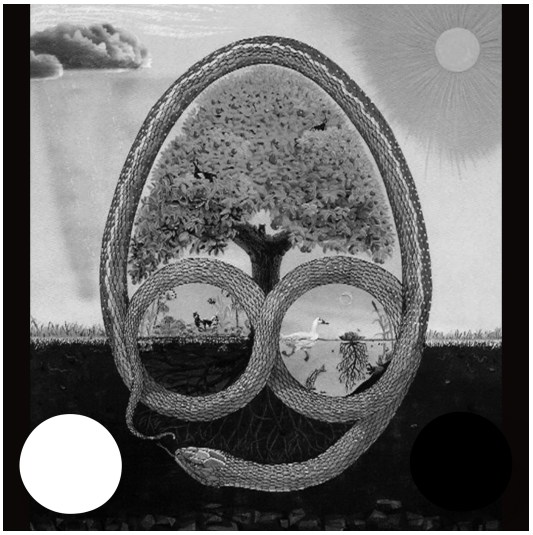
Phase 3: Proof verification in browser

- Implement `verifyMerkleProof()` in our uncloseai.js client
- Surface proof status in our chat UI (verified badge vs. fresh inference)
- Add document root display so users can audit our source fingerprint

Citation

```
russell@unturf, cthegray, TimeHexOn, foxhop.
"Merkle Providence Reverse RAG: Provenance-Preserving Cache Chains
for Small Language Models."
unfirehose.com, 2026.
https://unfirehose.com/merkle-providence-reverse-rag.html
```

License



GNU Affero General Public License v3

```
AGPL-3.0-only

PERMACOMPUTER PREAMBLE - NO WARRANTY

This is free software for the public good of a permacomputer hosted at
permacomputer.com - an always-on computer by the people, for the people. One
which is durable, easy to repair, and distributed like tap water for machine
learning intelligence.

The permacomputer is community-owned infrastructure optimized around four values:

  TRUTH    - Source code must be open source & freely distributed
  FREEDOM  - Voluntary participation without corporate control
  HARMONY  - Systems operating with minimal waste that self-renew
  LOVE     - Individual rights protected while fostering cooperation

This paper contributes to that vision by documenting Merkle Providence Reverse
RAG: a provenance-preserving cache layer that lets small language models punch
far outside their parameter class. Code is seeds to sprout on any abandoned
technology.

Learn more: https://www.permacomputer.com

NO WARRANTY. THE SOFTWARE IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND.

Copyright (C) 2026 TimeHexOn & foxhop & cthegray & russell@unturf
https://www.timehexon.com
https://www.foxhop.net
https://carltonthegray.com
https://www.unturf.com/software
https://www.permacomputer.com
https://uncloseai.com
https://unfirehose.com
https://russell.ballestrini.net

------------------------------------------------------------------------------

                     GNU AFFERO GENERAL PUBLIC LICENSE
                        Version 3, 19 November 2007

 Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
 Everyone is permitted to copy and distribute verbatim copies
 of this license document, but changing it is not allowed.
```